🗺️ Cartographer-SLAM Mapping

1. Course Content

- Master using Google Cartographer algorithm for 2D laser SLAM mapping.
- Master remote controlling the robot for mapping and viewing submap and global loop closure optimization effects in RViz2.
- Master methods for saving standard OccupancyGrid raster maps (`.yaml` + `.pgm`) and Cartographer pose graph state files (`.pbstream`).

---

2. Cartographer Introduction

Cartographer is a 2D/3D SLAM system open-sourced by Google based on Graph-based SLAM framework:

- **Submap Construction (Front-End)**: Continuous multiple frames of LaserScan laser data are inserted through correlative scan matching (CSM) to construct local Submap submaps.
- **Loop Closure Detection and Back-End Optimization (Back-End)**: When the number of constructed submaps increases, the back-end performs loop closure constraint solving (Loop Closure) multi-threaded via Ceres solver, eliminating accumulated drift from odometry and scan matching.
- **Features and Advantages**: Extremely high mapping accuracy, strong loop closure correction ability, supports building highly consistent maps in large-scale indoor complex environments.

---

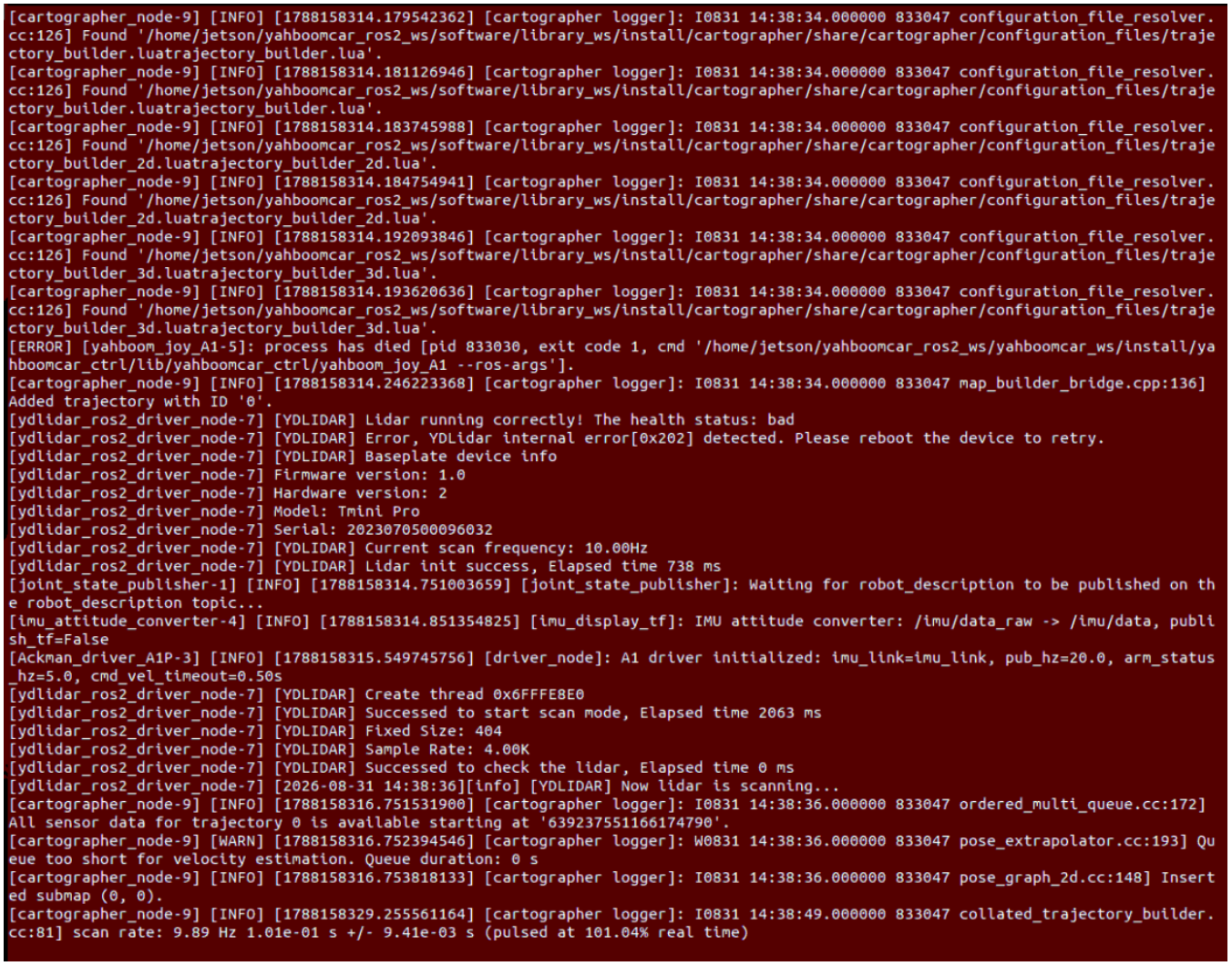
3. Preparation and Running

This example requires opening 3~4 terminals to start the following nodes in sequence:

3.1 Start Cartographer Mapping Node

Run in **Terminal 1** (automatically starts chassis driver, lidar driver, and Cartographer mapping core):

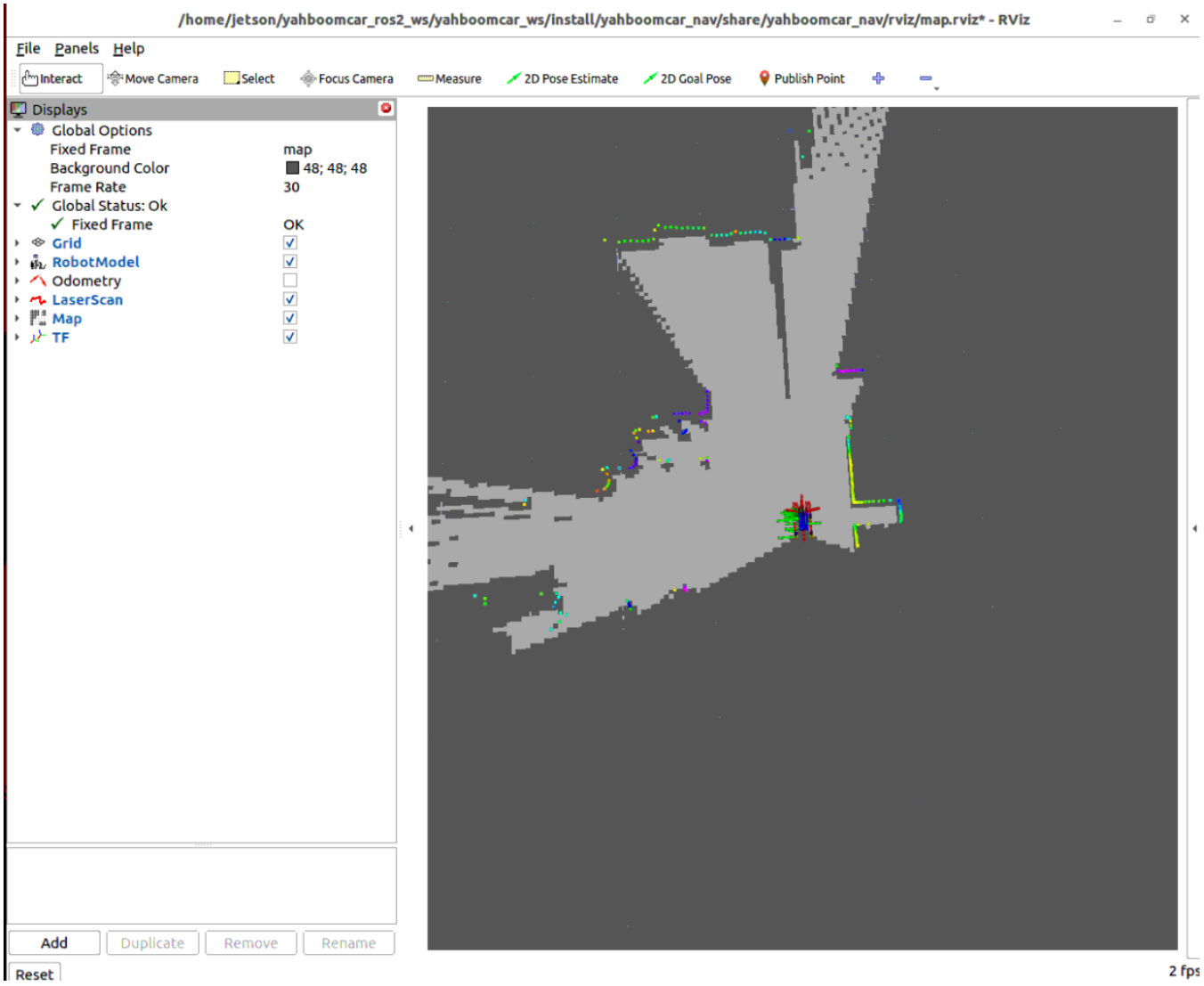
```
ros2 launch yahboomcar_nav map_cartographer_launch.py
```



3.2 Start RViz Map Visualization

Run in **Terminal 2** (it is recommended to open RViz on the virtual machine if the VM and robot are on the same LAN):

```
ros2 launch yahboomcar_nav display_map_launch.py
```



3.3 Start Keyboard Control to Move Robot for Mapping

Run in **Terminal 3** the keyboard control node:

```
ros2 run yahboomcar_ctrl yahboom_keyboard
```

```
jetson@yahboom:~/yahboomcar_ros2_ws/yahboomcar_ws$ ros2 run yahboomcar_ctrl yahboom_keyboard
1 - 6 : Select Joint 1~6
a / d : Increase (+) / Decrease (-) selected joint angle (step: 5 deg)
+ / - : Increase (+) / Decrease (-) selected joint angle (step: 5 deg)
0     : Reset all joints to initial pose

Angle Limits:
Joint 1~4 : 0° ~ 180°
Joint 5   : 0° ~ 270°
Joint 6   : 0° ~ 180°

anything else : stop smoothly
CTRL-C to quit

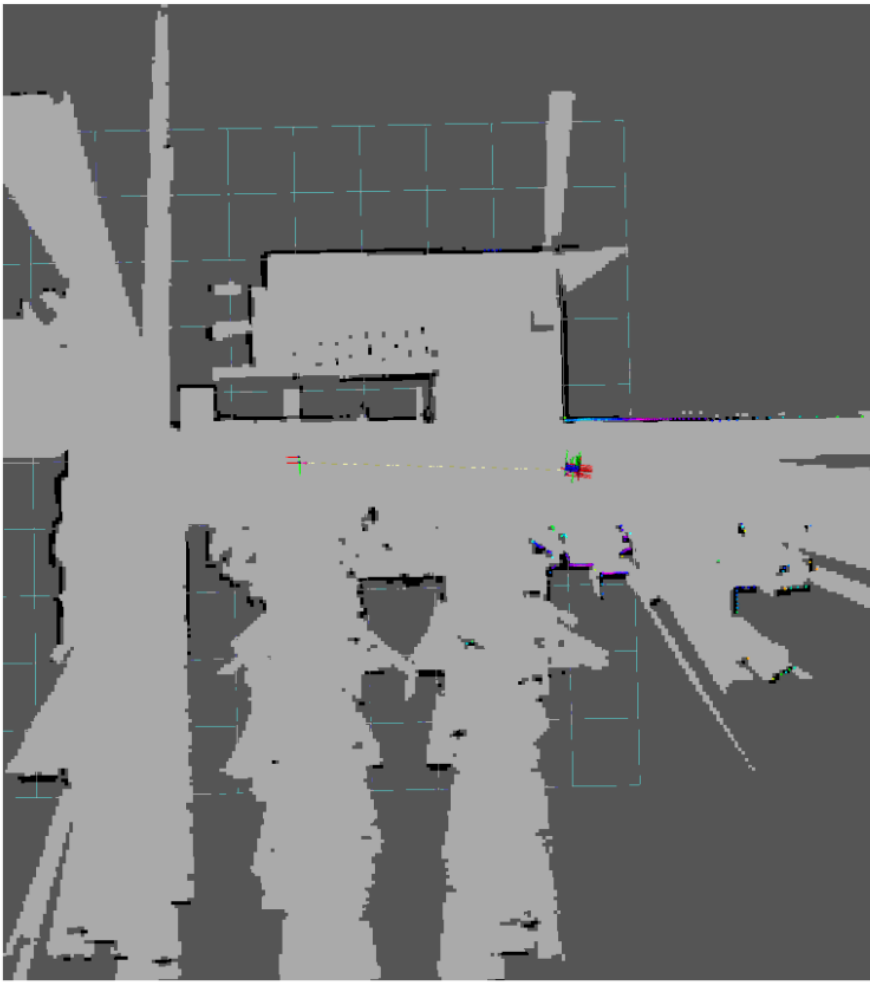
currently:      speed 0.2       turn 1.0
Current Joint Selection: Joint 1 (90°)
currently:      speed 0.18000000000000002       turn 0.9
currently:      speed 0.16200000000000003       turn 0.81
currently:      speed 0.14580000000000004       turn 0.7290000000000001
currently:      speed 0.13122000000000003       turn 0.6561000000000001
currently:      speed 0.11809800000000004       turn 0.5904900000000002
currently:      speed 0.12990780000000005       turn 0.6495390000000002
o
```

/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/install/yahboomcar_nav/share/yahboomcar_nav/rviz/map.rviz* - RViz

File Panels Help

Interact | Move Camera | Select | Focus Camera | Measure | 2D Pose Estimate | 2D Goal Pose | Publish Point

Displays
- Global Options
  - Fixed Frame: map
  - Background Color: 48; 48; 48
  - Frame Rate: 30
- Global Status: Ok
  - Fixed Frame: OK
- Grid
- RobotModel
- Odometry
- LaserScan
- Map
- TF



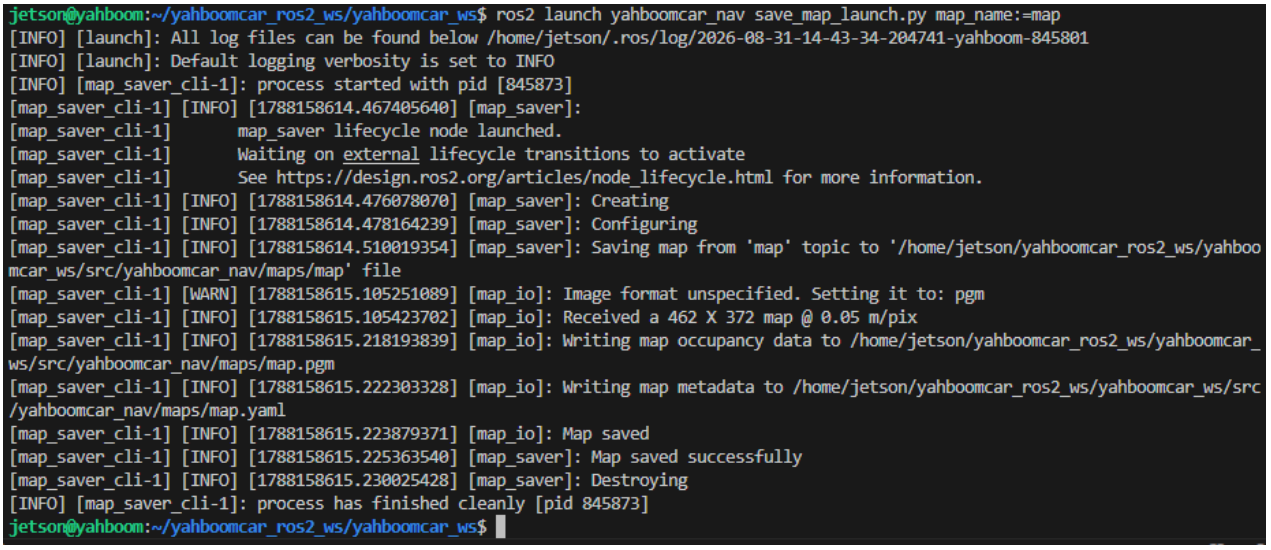
Add | Duplicate | Remove | Rename

Reset | **Left-Click:** Rotate. **Middle-Click:** Move X/Y. **Right-Click/Mouse Wheel:** Zoom. **Shift:** More options.

3.4 Mapping Operation and Map Save Instructions

1. **Mapping Driving Suggestions**:
   - Remote control the robot to slowly and smoothly drive through the entire environment, observe the continuous splicing of submaps in RViz.
   - When completing a loop back to the starting origin, the back-end will automatically match historical submaps and trigger global loop closure optimization, instantly eliminating corridor accumulated drift.
2. **Save 2D Raster Map (.yaml + .pgm)**:
   Run the map save command in **Terminal 4**:

   ```
   ros2 launch yahboomcar_nav save_map_launch.py map_name:=map
   ```

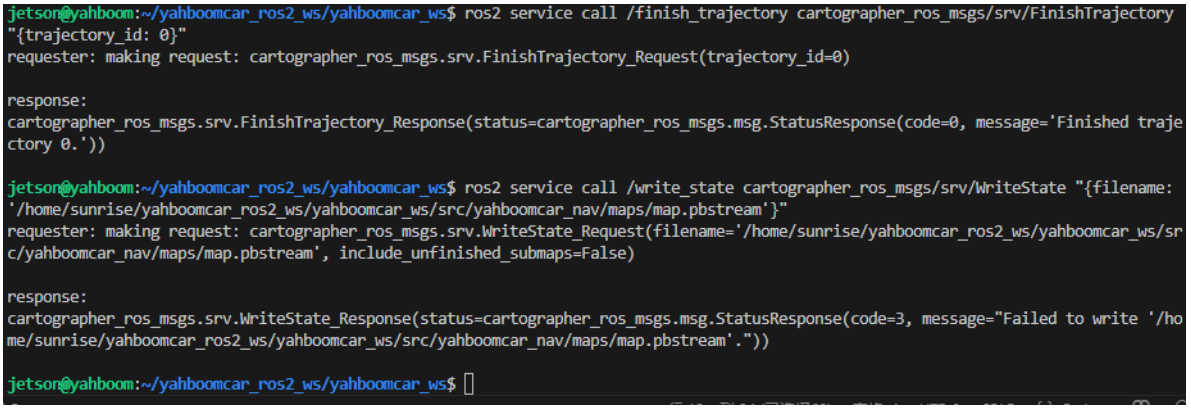
   

   Map files are saved in `~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/` directory.
3. **Save Cartographer Pose Graph File (.pbstream)**:
   The pose map (`.pbstream`) contains complete graph-optimized pose nodes and original constraint data, which is the required file for subsequent Cartographer pure localization relocalization navigation:
   - **Finish current mapping trajectory**:

     ```
     ros2 service call /finish_trajectory cartographer_ros_msgs/srv/FinishTrajectory "{trajectory_id: 0}"
     ```

   - **Write and save pose graph file**:

     ```
     ros2 service call /write_state cartographer_ros_msgs/srv/WriteState "{filename: '$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.pbstream'}"
     ```



> **Note**
>
> After saving is complete, press `Ctrl+C` in each terminal to stop the program.

---

4. Core Source Code and Algorithm Parameter Analysis

4.1 Mapping Launch File Analysis

Launch File Path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/launch/slam/map_cartographer_launch.py
```

```python
def generate_launch_description():
    # Introduce chassis and lidar driver
    laser_bringup_launch = IncludeLaunchDescription(PythonLaunchDescriptionSource(
        [os.path.join(get_package_share_directory('yahboomcar_nav'), 'launch','robot'),
         '/laser_bringup_no_odom_launch.py'])
    )

    # Introduce Cartographer SLAM mapping node
    cartographer_launch = IncludeLaunchDescription(PythonLaunchDescriptionSource(
        [os.path.join(get_package_share_directory('yahboomcar_nav'), 'launch','slam'),
         '/cartographer_launch.py'])
    )

    return LaunchDescription([laser_bringup_launch, cartographer_launch])
```

- `laser_bringup_no_odom_launch.py`: Start A1 Pro chassis and lidar driver, avoiding conflicts between external TF and Cartographer's own output TF.
- `cartographer_launch.py`: Load Cartographer core node `cartographer_node` and raster conversion node `occupancy_grid_node`.

4.2 Cartographer Algorithm Configuration and Core Parameter Interpretation

Configuration File Path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/params/lds_2d.lua
```

Core Parameter Description Table

Parameter Name	Current Config Value	Core Function and Physical Meaning
`tracking_frame`	`"base_footprint"`	Base coordinate frame Cartographer uses to track robot pose
`published_frame`	`"base_footprint"`	Sub-coordinate frame published by Cartographer (publishes `map -> base_footprint`)
`provide_odom_frame`	`true`	Whether Cartographer itself provides and publishes odometry TF relationship
`use_odometry`	`false`	Whether to subscribe to external wheel odometry data
`use_online_correlative_scan_matching`	`true`	Enable real-time correlative scan matching (CSM), greatly enhancing front-end matching robustness without prior
`min_range` / `max_range`	`0.1 / 10.0`	Shortest and longest effective filtered distance of lidar for mapping (meters)
`missing_data_ray_length`	`0.5`	Distance for inserting no-echo rays into free space (meters)
`constraint_builder.min_score`	`0.70`	Loop closure detection minimum matching score threshold, filtering low-confidence incorrect loop matches
`global_localization_min_score`	`0.70`	Global relocalization minimum matching score threshold

Lua Configuration File Core Source

```
include "map_builder.lua"
include "trajectory_builder.lua"

options = {
  map_builder = MAP_BUILDER,
  trajectory_builder = TRAJECTORY_BUILDER,
```

```
  map_frame = "map",
  tracking_frame = "base_footprint",
  published_frame = "base_footprint",
  odom_frame = "odom",
  provide_odom_frame = true,
  publish_frame_projected_to_2d = false,
  use_odometry = false,
  num_laser_scans = 1,
  lookup_transform_timeout_sec = 0.2,
  submap_publish_period_sec = 0.3,
  pose_publish_period_sec = 5e-3,
  trajectory_publish_period_sec = 30e-3,
}

MAP_BUILDER.use_trajectory_builder_2d = true

TRAJECTORY_BUILDER_2D.min_range = 0.1
TRAJECTORY_BUILDER_2D.max_range = 10.0
TRAJECTORY_BUILDER_2D.missing_data_ray_length = 0.5
TRAJECTORY_BUILDER_2D.use_imu_data = false
TRAJECTORY_BUILDER_2D.use_online_correlative_scan_matching = true
TRAJECTORY_BUILDER_2D.motion_filter.max_angle_radians = math.rad(0.1)

-- Ceres loop closure graph optimization matching threshold
POSE_GRAPH.constraint_builder.min_score = 0.7
POSE_GRAPH.constraint_builder.global_localization_min_score = 0.7

return options
```